Bug Report

Restful Booker API — Local Instance (localhost:3001)
QA Take-Home Assessment — Part 2: API Test Automation
Author: Uma Uka | Date: 30 August 2026

All defects below were found by the automated API test suite in `/api-tests` and independently reproduced via direct HTTP requests against a local Restful Booker instance (`docker run -d -p 3001:3001 mwinteringham/restfulbooker`). Each is tracked in the automated suite via Playwright's `test.fail()`, which keeps the assertion strict (encoding the spec-correct expected behavior) rather than weakening it to force a pass — see the README's known-bug policy for how this interacts with CI. Bug IDs below are placeholders (BUG-1..6) pending assignment in the team's formal tracker.

Summary

ID	Severity	Endpoint	Summary
BUG-1	**Low**	DELETE /booking/{id}	Returns 201 Created instead of 200/204 on a successful delete.
BUG-2	**Low**	DELETE /booking/{id}	Returns 405 instead of 404 when the booking is already deleted / missing.
BUG-3	**High**	POST /booking	Missing required field causes a 500 Internal Server Error instead of 400.
BUG-4	**High**	POST /booking	Wrong-typed field is silently coerced to null instead of rejected.
BUG-5	**Medium**	POST /booking	Inverted checkin/checkout date range is accepted without validation.
BUG-6	**High**	GET /booking (filter)	Date-range filtering does not correctly match bookings under any tested case.

Not a defect: SQL-injection-style input in text fields is handled safely (verbatim round-trip, no execution/corruption/5xx) — see the appendix at the end of this report.

BUG-1 — DELETE /booking/{id} returns 201 Created instead of 200/204 on success

Low

Endpoint

`DELETE /booking/{id}`

Preconditions

A valid auth token (POST /auth) and an existing booking id.

Steps to Reproduce

1. POST /auth with valid admin credentials to obtain a token.
2. POST /booking with a valid payload to create a booking; note the returned bookingid.
3. Send DELETE /booking/{bookingid} with header `Cookie: token=<token>`.
4. Observe the HTTP status code returned.

Request

```
DELETE /booking/52
Cookie: token=<token>
```

Expected Result

HTTP 200 OK or 204 No Content, per REST conventions for a successful delete.

Actual Result

HTTP 201 Created, with plain-text body "Created".

Why This Is a Genuine Defect (not expected/documented behavior)

201 Created means, per RFC 7231 section 6.3.2, “a new resource was created as a result of this request.” Returning it for a DELETE directly contradicts HTTP status-code semantics. This is not a documented or intentional API design choice — it could mislead strict HTTP-compliant clients or automated tooling that checks status codes to confirm deletion.

Automated Test Coverage

api-tests/tests/crud.spec.js — “DELETE returns a REST-correct success status” (tracked via test.fail()).

BUG-2 — DELETE on an already-deleted / non-existent booking returns 405 instead of 404

Low

Endpoint

`DELETE /booking/{id}`

Preconditions

A valid auth token; a booking id known not to exist (already deleted, or never created).

Steps to Reproduce

1. POST /auth to obtain a valid token.
2. POST /booking to create a booking, then DELETE it once successfully.
3. Send DELETE /booking/{same id} again with the same valid token.

4. Observe the HTTP status code. (Also reproducible directly against any id that was never created, e.g. DELETE /booking/99999999.)

Request

```
DELETE /booking/54
Cookie: token=<token>

(booking 54 was already deleted in a prior request)
```

Expected Result

HTTP 404 Not Found.

Actual Result

HTTP 405 Method Not Allowed, with plain-text body "Method Not Allowed".

Why This Is a Genuine Defect (not expected/documented behavior)

DELETE is clearly a supported method on this route — it succeeds for real resources. Returning 405 (“this method is not allowed on this resource”) for a missing resource sends the wrong signal; the correct code is 404. This could break client-side idempotent-delete retry logic that specifically checks for 404 to treat a delete as already completed.

Automated Test Coverage

api-tests/tests/crud.spec.js — “deleting an already-deleted booking returns 404” (tracked via test.fail()).

BUG-3 — POST /booking with a missing required field returns 500 instead of 400

High

Endpoint

`POST /booking`

Preconditions

None.

Steps to Reproduce

1. Send POST /booking with a JSON body omitting the required firstname field (see request below).
2. Observe the HTTP status code and response body.
3. Confirm via GET /booking that no new booking was actually persisted despite the error.

Request

```
POST /booking
Content-Type: application/json

{
  "lastname": "NoFirst",
  "totalprice": 100,
  "depositpaid": true,
  "bookingdates": {
    "checkin": "2026-10-01",
    "checkout": "2026-10-05"
  }
}
```

Expected Result
HTTP 400 Bad Request with a validation error identifying the missing field.

Actual Result
HTTP 500 Internal Server Error, with plain-text body "Internal Server Error". (Booking count confirmed unchanged before/after — no partial record is persisted.)

Why This Is a Genuine Defect (not expected/documented behavior)
Malformed client input is a routine, expected case any client can trigger by accident. It should be rejected with a 4xx validation error. A 500 indicates an unhandled exception in server-side request processing — a robustness defect, not intentional behavior — and could leak internal error details to a client in a less locked-down deployment.

Automated Test Coverage
api-tests/tests/negative-boundary.spec.js — “rejects create with a missing required field” (tracked via test.fail()).

BUG-4 — POST /booking silently coerces a wrong-typed field to null instead of rejecting it

High

Endpoint
`POST /booking`

Preconditions
None.

Steps to Reproduce
1. Send POST /booking with totalprice as a string instead of a number (see request below).
2. Observe the HTTP status code.
3. Inspect the totalprice value in the response body.

Request
```
POST /booking
Content-Type: application/json

{
  "firstname": "Type",
  "lastname": "Test",
  "totalprice": "not-a-number",
  "depositpaid": true,
  "bookingdates": {
    "checkin": "2026-11-01",
    "checkout": "2026-11-05"
  },
  "additionalneeds": "none"
}
```

Expected Result
HTTP 400 Bad Request, rejecting the invalid type.

Actual Result
HTTP 200 OK; the booking is created successfully but totalprice is silently stored and returned as null.

Why This Is a Genuine Defect (not expected/documented behavior)

The API accepts invalid input and silently corrupts the value instead of rejecting it. The client receives a 200 success response with no indication its data was dropped — the booking now has no price on record. This is a silent data-integrity defect, more dangerous than an outright rejection because it fails without any error signal.

Automated Test Coverage
api-tests/tests/negative-boundary.spec.js — “rejects create with a wrong-typed field” (tracked via test.fail()).

BUG-5 — POST /booking accepts an inverted date range (checkout before checkin)

Medium

Endpoint
`POST /booking`

Preconditions
None.

Steps to Reproduce

1. Send POST /booking with bookingdates.checkin later than bookingdates.checkout (see request below).
2. Observe the HTTP status code and the returned booking's dates.

Request

```
POST /booking
Content-Type: application/json

{
  "firstname": "Date",
  "lastname": "Test",
  "totalprice": 100,
  "depositpaid": true,
  "bookingdates": {
    "checkin": "2026-11-10",
    "checkout": "2026-11-01"
  },
  "additionalneeds": "none"
}
```

Expected Result
HTTP 400 Bad Request, rejecting the logically invalid date range.

Actual Result
HTTP 200 OK; the booking is created with the inverted range stored exactly as submitted.

Why This Is a Genuine Defect (not expected/documented behavior)
There is no domain-level validation ensuring checkout falls after checkin. Any downstream system computing a stay length, nightly rate, or invoice from this data would silently produce a negative or nonsensical value. This is a validation gap, not an intentionally permissive design — there is no legitimate reason a booking system would allow this.

Automated Test Coverage
api-tests/tests/negative-boundary.spec.js — “rejects create where checkout is before checkin” (tracked via test.fail()).

BUG-6 — GET /booking date-range filtering does not correctly filter results

High

Endpoint

`GET /booking?checkin={date}&checkout;={date}`

Preconditions

At least one known booking with a specific checkin/checkout date range.

Steps to Reproduce

1. Create a booking with a known date range, e.g. checkin=2026-12-01, checkout=2026-12-05; note its bookingid.
2. Send GET /booking?checkin=2026-12-01 and observe the result.
3. Send GET /booking?checkout=2026-12-05 and observe the result.
4. Send GET /booking?checkin=2026-12-01&checkout=2026-12-05 and observe the result.
5. Send GET /booking?checkin=2020-01-01&checkout=2030-01-01 (a range wide enough to include every booking in the system) and observe the result.

Request

```
GET /booking?checkin=2026-12-01
GET /booking?checkout=2026-12-05
GET /booking?checkin=2026-12-01&checkout=2026-12-05
GET /booking?checkin=2020-01-01&checkout=2030-01-01
```

Expected Result

Each query returns exactly the bookings whose dates fall within the specified range. At minimum, the wide range in step 5 should return every existing booking, including the one created in step 1.

Actual Result

Step 2 (checkin only): always returns an empty array [], even for the exact date of a known booking. Step 3 (checkout only): ignores the filter entirely and returns every booking in the system regardless of date. Step 4 (both params): returns an inconsistent, incomplete subset that excludes the booking from step 1 despite its dates matching exactly. Step 5 (wide range): still excludes several bookings that unambiguously fall within the range.

Why This Is a Genuine Defect (not expected/documented behavior)

This is not an edge case — the feature fails to filter correctly under every combination tested, including a deliberately over-inclusive range that should trivially return all data. Date-range search is a core, advertised capability of this endpoint; it is non-functional.

Automated Test Coverage

api-tests/tests/filtering.spec.js — “GET /booking filtered by checkin/checkout date range returns exactly the matching bookings” (tracked via test.fail()).

Appendix: Verified-Safe Behavior (Not a Defect)

SQL-injection-style input in text fields is handled safely

Endpoint

`POST /booking`

Steps

1. Send POST /booking with firstname set to Robert'); DROP TABLE bookings;--
2. Inspect the firstname value in the create response.
3. Send GET /booking/{id} and inspect the firstname value again.

Result

HTTP 200 OK; the value is stored and returned verbatim on both the create response and the subsequent GET, with no execution, no corruption, and no 5xx error. The service remained healthy (GET /ping) afterward.

Conclusion

No injection vulnerability was found in this field under this payload. Recorded here for completeness and as negative-test evidence, not as a defect.

Automated Test Coverage

api-tests/tests/negative-boundary.spec.js — “handles SQLi-style input safely without executing or corrupting it” (expected to pass — not tracked as a bug).